



KONGU ENGINEERING COLLEGE

(Autonomous)

Perundurai, Erode – 638060



DEPARTMENT OF COMPUTER APPLICATIONS

ADVANCED JAVA PROGRAMMING

(24MCT21)

SEMINAR

On

Java Collection Frameworks

Abstract

The Java Collection Framework (JCF) is a unified architecture in Java that provides a set of interfaces, classes, and algorithms to efficiently store and manage groups of objects. It is part of the java.util package and was introduced to overcome the limitations of traditional data structures like arrays. Unlike arrays, collections can dynamically grow or shrink, making them more flexible for handling large and changing data in modern applications.

The framework is built on core interfaces such as Collection, List, Set, Queue, and Map, each representing different types of data structures. These interfaces are implemented by classes like ArrayList, LinkedList, HashSet, TreeSet, and HashMap, each offering unique features in terms of ordering, duplication, and performance. The use of generics in the framework ensures type safety and helps prevent runtime errors, making programs more reliable and easier to maintain. It provides built-in algorithms through the Collections class for operations like sorting, searching, and reversing data. This reduces coding complexity. Overall, it simplifies data handling, promotes code reusability, and enables developers to build efficient and scalable applications.

Submitted By

Aishwarya R (25MCR007)

The background is a light gray gradient with several realistic water droplets of various sizes scattered across it. A faint, circular watermark is visible in the upper center, containing the text "www.civildatas.com".

JAVA COLLECTION FRAMEWORKS

AISHWARYA R

25MCR007

JAVA COLLECTION FRAMEWORKS:

- ❖ The java collection framework (JCF) is a set of classes and interfaces in java that helps to store, manage, and manipulate groups of objects.
- ❖ They provide ready made data structures for effective data storage and manipulation.
- ❖ Components of Java Collection Frameworks:
 - Interfaces (list, set, map, queue)
 - Classes (arraylist, hashset, hashmap, etc.)
 - Algorithms (sorting, searching)
- ❖ All of these are part of the java.util package.

LIST INTERFACE

- ❖ The list interface represents an ordered collection that allows duplicate elements. Elements can be accessed using an index.
- ❖ Classes implementing list interface:

1. ArrayList :

- ❖ Arraylist is a dynamic array that can grow and shrink automatically.

2. LinkedList:

- ❖ A linked list is a linear data structure where elements are stored in nodes, and each node contains two parts:
 - **Data** – stores the actual value
 - **Reference (pointer)** – stores the address of the next node

EXAMPLE

```
import java.util.arraylist;

import java.util.linkedlist;

public class main {

    public static void main(string[] args) {

        // arraylist example

        arraylist<string> a = new arraylist<>();

        a.add("apple");

        a.add("banana");

        system.out.println("arraylist: " + a);

        linkedlist<string> l = new linkedlist<>();
```

```
l.add("red");

l.add("green");

system.out.println("linkedlist: " + l);
```

Output:

ArrayList: [Apple, Banana]

LinkedList: [Red, Green]

SET INTERFACE

- ❖ The set interface is a collection that stores unique elements and does not allow duplicate values.

- ❖ Classes implementing set interface

1. HashSet class:

- ❖ HashSet is a class that implements set interface using a hash table.

2. LinkedHashSet class:

- ❖ LinkedHashSet is a class that implements set interface and maintains insertion order.

3. TreeSet class:

- ❖ TreeSet is a class that stores elements in sorted order.

EXAMPLE

```
import java.util.hashset;
import java.util.linkedhashset;
import java.util.treeset;

public class main {

    public static void main(string[] args) {

        hashset<integer> h = new hashset<>();
        h.add(3);
        h.add(1);
        h.add(2);

        linkedhashset<integer> lh = new linkedhashset<>();
        lh.add(3);
        lh.add(1);
        lh.add(2);
```

```
        treeset<integer> t = new treeset<>();
        t.add(3);
        t.add(1);
        t.add(2);

        system.out.println("hashset: " + h);
        system.out.println("linkedhashset: " + lh);
        system.out.println("treeset: " + t);

    }
}
```

Output:

HashSet: [1, 2, 3]

LinkedHashSet: [3, 1, 2]

TreeSet: [1, 2, 3]

QUEUE INTERFACE

- ❖ Queue is a collection that follows the FIFO principle (first in first out).
- ❖ The element added first will be removed first.
- ❖ Classes implementing queue interface:

1. PriorityQueue class:

- ❖ PriorityQueue stores elements based on **priority order**, not insertion order.
- ❖ Smallest element gets highest priority by default.
- ❖ Elements are automatically sorted.

EXAMPLE

```
import java.util.priorityqueue;

public class main {

    public static void main(string[] args) {

        priorityqueue<integer> queue = new
priorityqueue<>();

        queue.add(30);

        queue.add(10);

        queue.add(20);

        system.out.println("priorityqueue elements:
" + queue);
```

```
system.out.println("element with highest
priority: " + queue.peek());

    }

}
```

Output:

PriorityQueue elements: [10, 30, 20]

Element with highest priority: 10

MAP INTERFACE

❖ Map is a collection that stores data in key–value pairs.

❖ Each element has:

- Key - unique identifier
- Value - data associated with the key.

❖ Classes implementing map interface

1. **HashMap:** Stores elements using a hash table.

2. **LinkedHashMap:** Stores elements in insertion order.

3. **TreeMap:** Stores elements in sorted order of keys.

EXAMPLE

```
import java.util.hashmap;  
  
import java.util.linkedhashmap;  
  
import java.util.treemap;  
  
public class main {  
    public static void main(string[] args) {  
        hashmap<integer, string> hashmap = new hashmap<>();  
        hashmap.put(2, "banana");  
        hashmap.put(1, "apple");  
        hashmap.put(3, "mango");  
  
        linkedhashmap<integer, string> linkedhashmap = new  
linkedhashmap<>();  
        linkedhashmap.put(2, "banana");  
        linkedhashmap.put(1, "apple");  
        linkedhashmap.put(3, "mango");
```

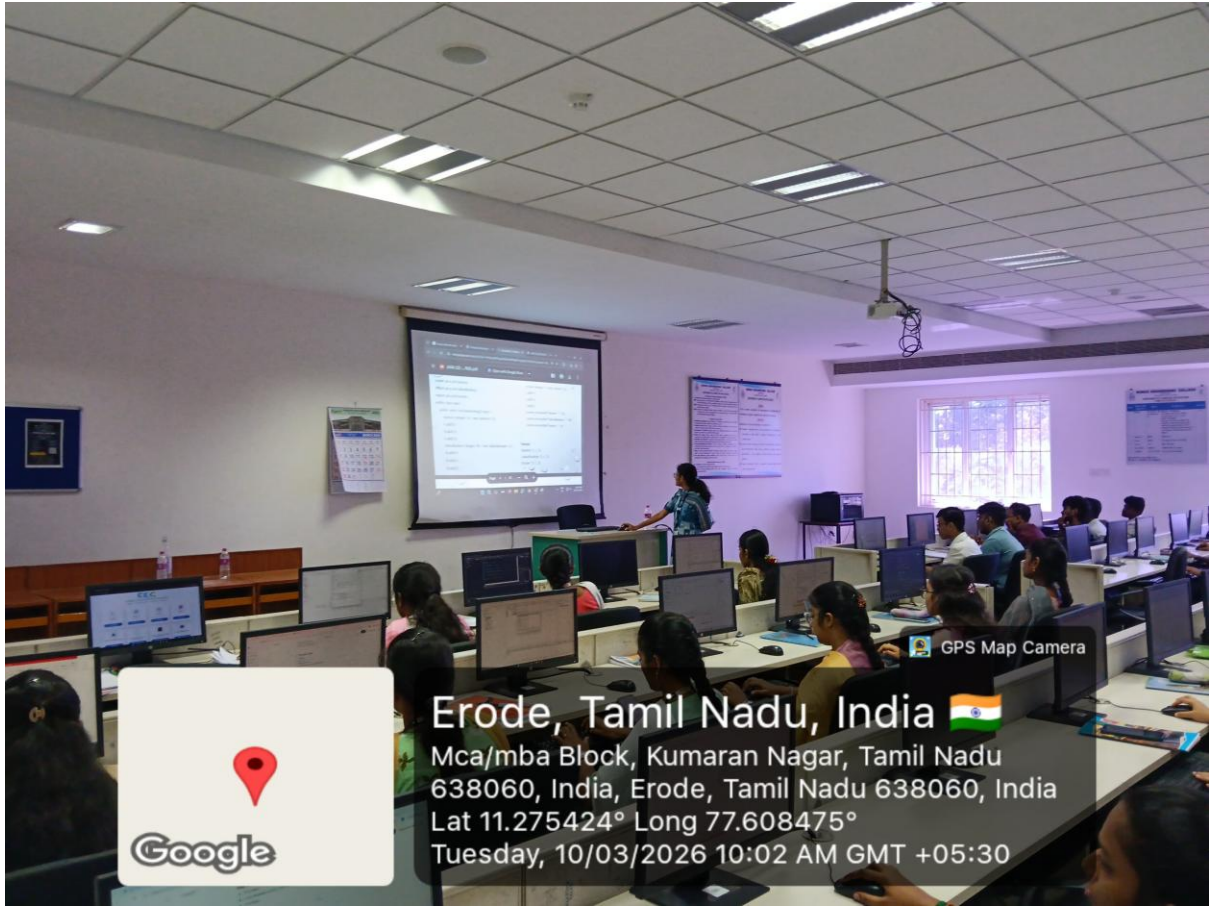
```
        treemap<integer, string> treemap = new  
treemap<>();  
        treemap.put(2, "banana");  
        treemap.put(1, "apple");  
        treemap.put(3, "mango");  
        system.out.println("hashmap: " + hashmap);  
        system.out.println("linkedhashmap: " +  
linkedhashmap);  
        system.out.println("treemap: " + treemap);  
    }  
}
```

Output:

HashMap: {1=Apple, 2=Banana, 3=Mango}

LinkedHashMap: {2=Banana, 1=Apple, 3=Mango}

TreeMap: {1=Apple, 2=Banana, 3=Mango}



Erode, Tamil Nadu, India 🇮🇳

Mca/mba Block, Kumaran Nagar, Tamil Nadu
638060, India, Erode, Tamil Nadu 638060, India

Lat 11.275424° Long 77.608475°

Tuesday, 10/03/2026 10:02 AM GMT +05:30